untangle::async

Generated by Doxygen 1.18.0

1 async 1

2 Topic Index 7
2.1 Topics . . . 7

3 Namespace Index 9
3.1 Namespace List . . . 9

4 Class Index 11
4.1 Class List . . . 11

5 File Index 13
5.1 File List . . . 13

6 Topic Documentation 15
6.1 namespace untangle: functions . . . 15

7 Namespace Documentation 17
7.1 untangle Namespace Reference . . . 17
7.1.1 Detailed Description . . . 17

8 Class Documentation 19
8.1 untangle::async::attachment Struct Reference . . . 19
8.1.1 Detailed Description . . . 19
8.2 untangle::async::execution< actionT > Class Template Reference . . . 19
8.2.1 Detailed Description . . . 21
8.2.2 Constructor & Destructor Documentation . . . 21
8.2.2.1 execution() [1/2] . . . 21
8.2.2.2 execution() [2/2] . . . 22
8.2.2.3 ~execution() . . . 22
8.2.3 Member Function Documentation . . . 22
8.2.3.1 create_instance() . . . 22
8.2.3.2 bind_action_and_method() . . . 23
8.2.3.3 bind_action_and_function() . . . 23
8.2.3.4 is_running() . . . 24
8.2.3.5 is_busy() . . . 24
8.2.3.6 pending() . . . 25
8.2.3.7 run() . . . 25
8.2.3.8 start() . . . 25
8.2.3.9 stop() . . . 26
8.2.3.10 add_action() . . . 26
8.2.3.11 add_task() . . . 27
8.2.3.12 add_queued_task() . . . 28
8.2.3.13 attach() . . . 28
8.2.3.14 detach() . . . 29
8.2.3.15 results() . . . 30

8.2.4 Member Data Documentation . . . 30
8.2.4.1 action_execute . . . 30
8.2.4.2 action_stop . . . 30
8.2.4.3 action_is_running . . . 31
8.2.4.4 on_finished . . . 31
8.2.4.5 on_error . . . 31
8.3 untangle::async::execution_poll Class Reference . . . 32
8.3.1 Detailed Description . . . 32
8.3.2 Constructor & Destructor Documentation . . . 32
8.3.2.1 ~execution_poll() . . . 32
8.3.3 Member Function Documentation . . . 33
8.3.3.1 add() . . . 33
8.3.3.2 remove() . . . 33
8.3.3.3 withdraw() . . . 33
8.3.3.4 is_running() . . . 34
8.4 untangle::async::invalid_attachment Struct Reference . . . 34
8.4.1 Detailed Description . . . 34
8.4.2 Constructor & Destructor Documentation . . . 34
8.4.2.1 invalid_attachment() . . . 34
8.4.3 Member Function Documentation . . . 35
8.4.3.1 what() . . . 35

9 File Documentation 37
9.1 async.hpp . . . 37

Index 47

Chapter 1

async

c++ mechanism to run queued callables in an asynchronous fashion

The interface consists of one header file that exposes a generic execution class. It is a wrapper of a queue of callables that are invoked on other thread. The callables are pumped into the queue by invoking an action from the caller thread.

The implementation is using the actuator callable, and therefore it exemplifies how a generic callable may improve the code structure, by easily creating interfaces inside a class instead of using external ones.

There are provided two execution modes: one-off and continuous.

requirements

C++23. The interface reports warnings with std::println, so it needs a standard library that provides <print> — GCC 14 or newer; older libc++ and MSVC releases may not have it yet.

one-off execution



example: one-off execution of a plain function

```
#include <async.hpp>

void f()
{
  std::cout « "f thread " « std::this_thread::get_id() « std::endl;
}

int main()
{
  std::cout « "main thread " « std::this_thread::get_id() « std::endl;

  // declare an execution specialized for an action type; it must be held by a shared pointer,
  // because an async binding keeps only weak ownership of it
  // the name is optional, and identifies the execution in the warnings it reports
  using void_execution = untangle::async::execution<std::function<void(void)»;
  auto execution = void_execution::create_instance("oneoff");
```

```
  std::function<void(void)> action;
  // create async binding between the action and f; the execution is passed as a shared pointer,
  // which the binding holds weakly - that is what lets the action outlive it safely
  void_execution::bind_action_and_function(action, f, execution);
  // whenever the action is invoked, a new callable wrapping f will be added to the execution's action queue
  action();
  // run the execution
  execution->run();

  // add the execution object to a poll of one's own; the poll answers for what was added to it
  untangle::async::execution_poll poll;
  poll.add(*execution);

  // wait the polled executions to finish
  while(poll.is_running())
  {
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
  }

  return 0;
}
```

example: one-off execution of a class method

```
#include <async.hpp>

class A
{
public:
  void f(int x)
  {
    std::cout « "A::f " « x « " thread " « std::this_thread::get_id() « std::endl;
  }
  std::function<void(int)> action;
};

int main()
{
  std::cout « "main thread " « std::this_thread::get_id() « std::endl;

  auto a = std::make_shared<A>();

  // declare an execution specialized for an action type; it must be held by a shared pointer
  using int_execution = untangle::async::execution<std::function<void(int)»;
  auto execution = int_execution::create_instance("oneoff_method");
  // note both the bound object and the execution must be shared pointers
  int_execution::bind_action_and_method(a->action, a, &A::f, execution);
  // whenever the action is invoked, a new callable wrapping f will be added to the execution's action queue
  a->action(10);
  // run the execution
  execution->run();

  // add the execution object to a poll of one's own; the poll answers for what was added to it
  untangle::async::execution_poll poll;
  poll.add(*execution);

  // wait the polled executions to finish
  while(poll.is_running())
  {
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
  }

  return 0;
}
```

continuous execution



example: continuous execution of a plain function

```
#include <async.hpp>

void f()
{
```

```
  std::cout « "f thread " « std::this_thread::get_id() « std::endl;
}

int main()
{
  std::cout « "main thread " « std::this_thread::get_id() « std::endl;

  // declare an execution specialized for an action type; it must be held by a shared pointer
  using void_execution = untangle::async::execution<std::function<void(void)»;
  auto execution = void_execution::create_instance("continuous");
  std::function<void(void)> action;
  // create async binding between the action and f
  void_execution::bind_action_and_function(action, f, execution);

  // run the execution
  execution->start();
  // add f to the execution's queue (three times)
  action();action();action();
  // stop the execution
  execution->stop();

  // add the execution object to a poll of one's own; the poll answers for what was added to it
  untangle::async::execution_poll poll;
  poll.add(*execution);

  // wait the polled executions to finish
  while(poll.is_running())
  {
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
  }

  return 0;
}
```

queueing and lifetime

Queueing is thread safe. An action may be invoked from any thread; the execution's list is guarded, and the worker is woken rather than polled for.

stop() ends the execution's working life. The worker runs whatever was queued before the call and then leaves. An action invoked after stop() is refused rather than queued, because it would never run, and the refusal is reported:

```
warning: execution 'continuous' is stopped, action not added
```

A task is refused after stop() for the same reason, and says so in its own words:

```
warning: execution 'continuous' is stopped, task not added
```

A task is also refused before it is ever queued if it could not do its job — no callback to notify, or nothing to run. That one is the caller's own mistake and is caught while they are still on the stack, rather than surfacing later as a failure of a task whose action had in fact succeeded:

```
warning: execution 'continuous' refused a task that cannot report
```

An action bound to an object that has since been destroyed is dropped the same way, rather than ending the process:

```
warning: execution 'continuous' dropped an invalid action: bind: invalid object
```

The worker is detached and cannot be joined. Waiting for an execution therefore means waiting on its running state, which is what execution_poll reports and what the examples above do. The destructor waits on the same state, so an execution that goes out of scope while its worker is still running blocks until the worker is finished rather than leaving it reading freed memory.

The poll holds a pointer to each execution added to it, so an execution withdraws itself in its destructor - from every poll it was added to, since it may be in more than one. A poll destroyed first likewise releases what it still holds, so a poll may be a local. execution_poll::remove() is available for withdrawing one earlier; calling it is not required.

tasks: work that reports back

Everything above queues actions, through add_action(). An action is fire and forget: it runs, what it returns is dropped, and the caller hears nothing more about it. A task, through add_task(), carries the callback it must notify:

```
auto exec = untangle::async::execution<std::function<int(int)»::create_instance("scale");
exec->start();

// the callback comes last, after the arguments the action is bound to
exec->add_task([](int n) { return n * 2; }, 21, [](int result) { /* result == 42 */ });
```

A task whose action returns void still reports that it finished, with a callback taking nothing — finished is the message and the result is optional:

```
exec->add_task([](int by) { /* ... */ }, 5, [] { /* finished */ });
```

The callback is the last argument, and the signature cannot say so. A parameter pack cannot be followed by a deducible parameter, so it arrives inside the arguments and untangle::bind_task() splits it off. It must be callable with the action's result and return nothing — or callable with nothing at all, when the action returns void. A missing or unusable one is a compile error rather than silence.

Why a task carries its own callback rather than being handed one. The actuator has a callback convention already: pass a trailing callable to an actuator's call operator and it is invoked with each action's result. That convention reads the trailing argument of an invocation, and a queued call has no invocation left to read from — so the callback has to travel with the work. That is what a task is.

add_task() answers bool, and the answer is the whole report. It refuses a task the execution can never run — because it is stopped — or one that could never do its job: no callback to notify, or nothing to run. A refused task is dropped, so it never runs and never notifies, and a caller waiting on the callback alone would wait for ever.

Finished does not mean failed. A task whose action throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to on_error, as a failing action's does. And because a callback runs inside the same try as the task that owns it, on_error can fire for a task whose action in fact succeeded — the work was done and only the telling failed.

on_finished is not a task's callback. It is raised once per pass, when the queue has drained, and a pass that ran only tasks raises it like any other. A task's own callback has already run by then, inside the same drain and on the same thread. Two notifications, two meanings: one per task, one per drained queue.

Both kinds share one queue and one worker. Within a single pass every action runs before any task, so the two do not interleave in the order they were queued; across passes the queue is still first in, first out.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

test/CMakeLists.txt fetches googletest at configure time, so the first configure needs network access. The actuator submodule has to be present: git submodule update --init.

sanitizers

ASYNC_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DASYNC_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

The two answer different questions. ThreadSanitizer names both sides of a data race, which is how the queue's races were diagnosed. AddressSanitizer reports a use-after-free, which is how the dangling attachment was. Neither sees everything: a defect reached through a destroyed std::mutex throws std::system_error from inside libsystem before any instrumented load runs, so both sanitizers stay silent. Reach for a direct instrumented read when that happens.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions 15

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle
Interface to untangle::async::execution class 17

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::async::attachment
: One execution's place in the attachment graph: who attached it, if anyone 19

untangle::async::execution< actionT >
: Asynchronous template execution class 19

untangle::async::execution_poll
: Poll that reports whether any execution added to it is still running 32

untangle::async::invalid_attachment
: Raised by execution::attach() when the attachment asked for is refused 34

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

async.hpp 37

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to untangle::async::execution class.

7.1.1 Detailed Description

Interface to untangle::async::execution class.

Chapter 8

Class Documentation

8.1 untangle::async::attachment Struct Reference

One execution's place in the attachment graph: who attached it, if anyone.

#include <async.hpp>

Public Attributes

- std::weak_ptr< attachment > attacher
  The link of the execution that attached this one; empty when nothing has.

8.1.1 Detailed Description

One execution's place in the attachment graph: who attached it, if anyone.

At namespace scope, so it is one type rather than one per specialisation: an execution may be attached by an execution of any specialisation, and the chain is walked without knowing any link's action type. It dies with its execution, so it doubles as a liveness token - a link that still locks belongs to an execution that is still there.

The documentation for this struct was generated from the following file:

- async.hpp

8.2 untangle::async::execution< actionT > Class Template Reference

Asynchronous template execution class.

#include <async.hpp>

Public Member Functions

- execution ()

  Constructs an execution with the default name. Call create_instance() instead.
- execution (std::string exec_name)

  Constructs a named execution. Use create_instance() for one that is to be bound.
- ~execution ()

  Destroys the execution, once its worker has left.
- bool is_running () const

  Is a worker of this execution's still there?
- bool is_busy () const

  Is this execution working through its action queue?
- std::size_t pending () const

  How many actions and tasks are queued and not yet taken by the worker.
- void run ()

  Runs what is queued on a worker of its own, once, and keeps the results.
- void start ()

  Starts a continuous worker that runs whatever is queued until stop().
- void stop ()

  Stops the worker, after the actions already queued have run.
- template<typename... Args>
  bool add_action (actionT action, Args &&... args)

  Queues an action, bound to args, and says whether it was taken.
- template<typename... Args>
  bool add_task (const actionT &action, Args &&... args)

  Queues a task, and says whether it was taken.
- template<typename... Args>
  bool add_task (actionT &&action, Args &&... args)

  Queues a task as the overload above does, moving the action into it rather than copying it. A temporary and a lambda land here.
- bool add_queued_task (untangle::task_t task)

  Queues a ready-made task, whatever this execution's action type, and says whether it was taken. What add_task() does once it has bound one.
- template<typename otherT>
  void attach (const std::shared_ptr< otherT > &other)

  Attaches another execution, so that this one triggers it.
- template<typename otherT>
  bool detach (otherT &other)

  Detaches an execution previously attached by attach().
- auto results () const

  The return values of the actions run by run(), in the order they ran.

Static Public Member Functions

- template<typename... Args>
  static std::shared_ptr< execution > create_instance (Args &&... args)

  Creates an execution owned by a std::shared_ptr, which is what binding to it requires.
- template<typename classT, typename T>
  static void bind_action_and_method (actionT &action, const std::shared_ptr< classT > &obj, T classT::*method, const std::shared_ptr< execution > &async_exec)

  Binds asynchronously an external action to a class function member.
- template<typename T>
  static void bind_action_and_function (actionT &action, T Fn, const std::shared_ptr< execution > &async_exec)

  Binds asynchronously an external action to a plain function.

Public Attributes

- std::function< void(void)> action_execute

  Connection point that runs this execution's queued actions. Bound to execute_actions().
- std::function< void(void)> action_stop

  Connection point that stops this execution. Bound to stop().
- std::function< bool(void)> action_is_running

  Connection point that reports whether this execution is running. Bound to is_running().
- std::function< void(void)> on_finished

  Called on the worker's thread once the queue has drained. Assigned by the caller.
- std::function< void(std::exception_ptr)> on_error

  Called on the worker's thread with whatever an action threw. Assigned by the caller.

Static Public Attributes

- static constexpr auto default_name = "default_name"

  Name of an execution built unnamed.

8.2.1 Detailed Description

template<typename actionT>
class untangle::async::execution< actionT >

Asynchronous template execution class.

An async execution provides a mechanism to queue work and execute it sequentially on a separate thread. It may also attach another execution object and trigger its actions. This way actions of different types may be executed on the same thread.

Two kinds of work may be queued. An action, through add_action(), is fire and forget: it runs, and the caller hears nothing more about it. A task, through add_task(), carries the callback it must notify, and reports what it produced - or, for an action returning void, merely that it finished. A queued call has no invocation left for the actuator's own callback convention to read a callback from, which is why a task carries its own.

The mechanism relies on an "asynchronous binding" created by bind_action_and_method() or bind_action_and_function().

Template Parameters

actionT	It represents the type of the action. It is specified as std::function<...> and should match the signature of the bound function or class method.

8.2.2 Constructor & Destructor Documentation

8.2.2.1 execution() [1/2]

template<typename actionT>
untangle::async::execution< actionT >::execution () [inline]

Constructs an execution with the default name. Call create_instance() instead.

The name identifies this execution in the warnings it reports, and is otherwise unused.

8.2.2.2 execution() [2/2]

template<typename actionT>
untangle::async::execution< actionT >::execution (
std::string exec_name) [inline], [explicit]

Constructs a named execution. Use create_instance() for one that is to be bound.

Parameters

exec_name	- A name for this execution.

8.2.2.3 ~execution()

template<typename actionT>
untangle::async::execution< actionT >::~execution () [inline]

Destroys the execution, once its worker has left.

The worker is detached and cannot be joined, so this waits on the "running" state instead, and returns at once for one that was never started. An attacher holds this execution weakly, so there is nothing to take back out of it: its next pass finds this gone and drops it.

Attention

Safe only because running_ is cleared as the very last thing the worker does: this object may be freed the moment it reads false, so nothing may follow it in execute() or loop().

Remarks

No rule of five, deliberately. An execution is declared once and used in place; the members already delete the copy operations several times over.

8.2.3 Member Function Documentation

8.2.3.1 create_instance()

template<typename actionT>
template<typename... Args>
std::shared_ptr< execution > untangle::async::execution< actionT >::create_instance (
Args &&... args) [inline], [static]

Creates an execution owned by a std::shared_ptr, which is what binding to it requires.

An execution built any other way is still usable - add_action() takes actions directly - but cannot be bound, and the attempt does not compile.

Parameters

args	- Constructor arguments: a name, or nothing for the default name.

Returns

- A std::shared_ptr owning the new execution.

8.2.3.2 bind_action_and_method()

```
template<typename actionT>
template<typename classT, typename T>
void untangle::async::execution< actionT >::bind_action_and_method (
        actionT & action,
        const std::shared_ptr< classT > & obj,
        T classT::* method,
        const std::shared_ptr< execution< actionT > > & async_exec)   [inline], [static]
```

Binds asynchronously an external action to a class function member.

It creates an action as a binding to a class method (by untangle::bind()), and assigns to action a callable that queues it. The execution is held weakly, so the action may outlive it; the action itself is shared with every call queued from it, rather than copied per call.

Attention

Invoking action after the execution has been destroyed throws untangle::invalid_action. The worker catches it and drops the action with a warning.

Parameters

action	[in,out] - An action of type std::function<...>.
obj	- A std::shared_ptr that wraps the bound class object.
method	- Pointer to function member. It is specified as &<class type>::<function member>.
async_exec	- A std::shared_ptr owning the execution the action is queued on. Requiring it here is what keeps an execution that no shared_ptr owns from being bound at all.

8.2.3.3 bind_action_and_function()

```
template<typename actionT>
template<typename T>
void untangle::async::execution< actionT >::bind_action_and_function (
        actionT & action,
        T Fn,
        const std::shared_ptr< execution< actionT > > & async_exec)   [inline], [static]
```

~~Binds asynchronously an external action to a plain function.~~

It wraps Fn in an action and assigns to action a callable that queues it. The execution is held weakly, as in bind_action_and_method(), and the action is shared with every call queued from it rather than copied per call.

Parameters

action	[in,out] - An action of type std::function<...>.
Fn	- A plain function, or any callable; moved into the binding.
async_exec	- A std::shared_ptr owning the execution the action is queued on.

8.2.3.4 is_running()

template<typename actionT>
bool untangle::async::execution< actionT >::is_running () const [inline]

Is a worker of this execution's still there?

True from run() or start() until the worker has left, which is the same instant ~execution() waits for and the same one run() waits for before starting another.

Remarks

on_finished is raised from inside the drain and reads this as true - the worker is standing in the callback with the rest of its path still to go.

Returns

true - a worker is still there.

8.2.3.5 is_busy()

template<typename actionT>
bool untangle::async::execution< actionT >::is_busy () const [inline]

Is this execution working through its action queue?

True from the moment an action is added until the last one has returned. Unlike is_running(), which is about the worker thread, this answers "has it room for more work"

- a worker started by start() is running for its whole life whether busy or idle.

Remarks

Answers for this execution's own actions, not for those it triggers through attach().

A false answer is durable only for the caller that is itself adding the actions; a true one may stop being true the moment after.

Returns

true - actions are queued, or one is running.

8.2.3.6 pending()

template<typename actionT>
std::size_t untangle::async::execution< actionT >::pending () const [inline]

How many actions and tasks are queued and not yet taken by the worker.

What the worker has taken into the batch it is running is not pending: a worker running its last action reports 0 here and true from is_busy().

Remarks

Advisory, like is_busy(): true the moment it is read.

8.2.3.7 run()

template<typename actionT>
void untangle::async::execution< actionT >::run () [inline]

Runs what is queued on a worker of its own, once, and keeps the results.

The worker is detached: wait for it with is_running() or execution_poll. Actions queued while it is draining are run by it too. results() is filled by this path only.

Blocks until a worker still resident from a previous run has left, so one execution never has two. is_running() reads false at exactly that moment, so a caller that waits for it and then runs again does not wait here at all.

Attention

Never call this from inside an action: the worker that would have to leave is the one making the call, so it waits for itself and hangs.

8.2.3.8 start()

template<typename actionT>
void untangle::async::execution< actionT >::start () [inline]

Starts a continuous worker that runs whatever is queued until stop().

Unlike run(), the worker stays for the life of the execution, so is_running() is true whether or not there is work; ask is_busy() instead. Results are not collected.

Blocks until a worker still resident has left, as run() does.

Attention

Never call this from inside an action, for the reason given on run().

8.2.3.9 stop()

```
template<typename actionT>
void untangle::async::execution< actionT >::stop ()  [inline]
```

Stops the worker, after the actions already queued have run.

Remarks

An action passed to add_action() after this call is refused, not queued: stop() is the end of this execution's working life, and an action accepted then would never run.

8.2.3.10 add_action()

```
template<typename actionT>
template<typename... Args>
bool untangle::async::execution< actionT >::add_action (
                actionT action,
                Args &&... args)  [inline]
```

Queues an action, bound to args, and says whether it was taken.

It runs later, on whichever thread drives this execution - its own worker, or an attacher's. Queueing before run() or start() is normal.

Attention

A stopped execution refuses and drops the action, silently for a caller that ignores the answer. Nothing is thrown: untangle::invalid_action means a dead binding, while a refused action is sound and merely too late.

Remarks

A caller arriving through bind_action_and_method() or bind_action_and_function() never sees the answer: those lambdas return actionT::result_type, which has no room for it.

It does not notify anyone. The answer below is the last the caller hears: what the action returns is run and dropped, and nothing reports that it finished. Use add_task() for work that has to report. A trailing callable passed here is an ordinary argument of the action - the actuator's callback convention reads a trailing argument of an invocation, and a queued call has none.

Template Parameters

Args	- The argument types the action is bound to.

Parameters

action	- The action to queue.
args	- The arguments to bind to action.

Returns

true - queued, and this execution's worker will run it.

false - the execution is stopped and the action was dropped.

8.2.3.11 add_task()

```
template<typename actionT>
template<typename... Args>
bool untangle::async::execution< actionT >::add_task (
                const actionT & action,
                Args &&... args)   [inline]
```

Queues a task, and says whether it was taken.

A task is an action bound to its arguments and to the callback it must notify - unlike add_action(), which queues work the caller hears nothing more about. It runs on whichever thread drives this execution, and the callback is invoked there too, with what the action returned. Queueing before run() or start() is normal.

Attention

The last argument is the callback, and this signature cannot say so: a parameter pack cannot be followed by a deducible parameter, so it arrives inside args and untangle::bind_task() splits it off. It must be callable with the action's result and return nothing - or callable with nothing at all, when the action returns void, because finished is the message and the result is optional. A missing or unusable one is a static_assert rather than silence.

Finished does not mean failed. A task whose action throws is not notified - there is no result to hand over - and what it threw goes to on_error, as a failing action's does. A caller waiting on the callback alone would wait for ever.

Remarks

The actions convention on untangle::actuator does not apply here and never did: it reads a trailing argument of an invocation, and a queued call has no invocation left to read from. That is what a task is for.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.

Parameters

action	- The action to run, copied into the task.
args	- The arguments to bind to action, followed by the callback to notify.

Returns

true - queued, and this execution's worker will run it and notify the callback.

false - refused and dropped, so it will never run and never notify. Either the execution is stopped, or the task could not do its job: no callback to notify, or nothing to run. The answer is the whole report - nothing is thrown and the callback is not invoked.

8.2.3.12 add_queued_task()

```
template<typename actionT>
bool untangle::async::execution< actionT >::add_queued_task (
                untangle::task_t task)  [inline]
```

Queues a ready-made task, whatever this execution's action type, and says whether it was taken. What add_task() does once it has bound one.

Meant for a pool built on executions, such as untangle::executor: its own code - an ordered queue's drain, which runs that queue's tasks in turn - is not of the action type, so add_task() cannot queue it.

A task that throws goes to on_error, as a failing action does.

Parameters

task	- The task to run: it does its job when called, and answers whether it did.

Returns

true - queued; false - the execution is stopped, or task is empty.

8.2.3.13 attach()

```
template<typename actionT>
template<typename otherT>
void untangle::async::execution< actionT >::attach (
                const std::shared_ptr< otherT > & other)  [inline]
```

Attaches another execution, so that this one triggers it.

The attached execution's pending actions are run by this one's worker, and stopping this one stops that one too. Actions of different types can be run on one thread this way.

other is held weakly: a pass locks it for the call, so it is never destroyed while this execution's worker is running it, and one that is gone by the next pass is dropped there.

Parameters

other	- A std::shared_ptr owning the execution to trigger; requiring one keeps an unowned execution from being attached at all.

Exceptions

invalid_attachment	- if other is attached already. An execution has at most one attacher, which keeps the graph a forest and the cycle check a walk rather than a search.
invalid_attachment	- if the attachment would close a cycle, itself included. Driving a cycle recurses until the stack is gone, so it is refused where the caller can still be told.

8.2.3.14 detach()

```
template<typename actionT>
template<typename otherT>
bool untangle::async::execution< actionT >::detach (
                otherT & other)   [inline]
```

Detaches an execution previously attached by attach().

Both paths are unwired together, the triggering one and the stopping one.

Parameters

other	- The execution to stop triggering. Detaching one that was never attached is not an error: the caller asked for a state that already holds.

Remarks

Refused, and nothing changed, from inside this execution's own attachment pass - from an action of other, say: its actuators are dispatching, and refuse removals then. Detach once the pass is over.

Returns

true - other was attached and is no longer.

false - other was not attached to this execution, or the detach was refused.

8.2.3.15 results()

template<typename actionT>
auto untangle::async::execution< actionT >::results () const [inline]

The return values of the actions run by run(), in the order they ran.

Read it from on_finished, which fires once the queue has drained, so the results are complete by the time the callback sees them. Returned by value: the worker is appending to it.

Remarks

Only run() fills this. A continuous worker has no point at which a run is over, so start() collects nothing.

Returns

The results of the most recent run, or empty if none has produced any.

8.2.4 Member Data Documentation

8.2.4.1 action_execute

template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_execute

Connection point that runs this execution's queued actions. Bound to execute_actions().

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.2 action_stop

template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_stop

Connection point that stops this execution. Bound to stop().

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.3 action_is_running

template<typename actionT>
std::function<bool(void)> untangle::async::execution< actionT >::action_is_running

Connection point that reports whether this execution is running. Bound to is_running().

execution_poll holds its address, and ~execution() takes it back out.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.4 on_finished

template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::on_finished

Called on the worker's thread once the queue has drained. Assigned by the caller.

results() is complete by the time it runs, and is_running() reads true there.

Remarks

It is raised once per pass, not once per piece of work, and a pass that ran only tasks raises it like any other. Do not read it as "my task finished": that is the task's own callback, and it has already run by this point, inside the same drain and on this thread.

A worker that woke with an empty queue has finished nothing and stays silent.

8.2.4.5 on_error

template<typename actionT>
std::function<void(std::exception_ptr)> untangle::async::execution< actionT >::on_error

Called on the worker's thread with whatever an action threw. Assigned by the caller.

The action is dropped either way; this is what it is dropped to. Without it the only record is a warning on stderr naming this execution, which reaches a log and no code — an action that failed and one that succeeded are the same from outside. A handler assigned here replaces that warning, so a caller that takes responsibility for reporting is not reported twice.

Remarks

It receives a std::exception_ptr because the worker catches everything, including what is not a std::exception. Rethrow it to read it.

It is told about a dead binding too, which arrives as untangle::invalid_action. A caller asking why an action did not run wants both answers, not the one the header happens to raise itself.

A task's failure arrives here as well, and so does its callback's. A task whose action throws is not notified - there is no result to hand over - so this is the only report it makes. And because a callback runs inside the same try as the task that owns it, this can fire for a task whose action in fact succeeded: the work was done, and only the telling failed.

Attention

It runs on the worker's thread, inside the drain, like on_finished. Nothing may escape it: it is called from the same try the action was called from, and an exception leaving a detached thread function calls std::terminate.

The documentation for this class was generated from the following file:

- async.hpp

8.3 untangle::async::execution_poll Class Reference

Poll that reports whether any execution added to it is still running.

#include <async.hpp>

Public Member Functions

- execution_poll ()=default

  Constructs an empty poll. add() is what puts anything in it.
- ~execution_poll ()

  Destroys the poll, releasing every execution still in it.
- template<typename asyncexecT>
  void add (asyncexecT &async_exec)

  Adds an execution object to the poll.
- template<typename asyncexecT>
  void remove (asyncexecT &async_exec)

  Removes an execution object from the poll.
- void withdraw (const std::function< bool(void)> *action)

  Takes one action pointer out, without touching the execution it came from.
- auto is_running ()

  Checks if the execution objects are running.

8.3.1 Detailed Description

Poll that reports whether any execution added to it is still running.

Remarks

An execution runs in a detached thread and cannot be joined, so checking the "running" state is the only way to wait for one. This checks any number of them at once.

A poll is constructed and owned by the caller, and answers for the executions added to that one. An owner of a set of workers - a thread pool is the case this was written for - waits for its own and not for whatever else the process is running.

8.3.2 Constructor & Destructor Documentation

8.3.2.1 ~execution_poll()

untangle::async::execution_poll::~execution_poll () [inline]

Destroys the poll, releasing every execution still in it.

Remarks

The record runs both ways, and it has to: each side holds a pointer into the other, so whichever dies first takes itself out of the other. ~execution() withdraws from every poll holding it; this tells every execution it holds that this poll is gone. Neither order leaves a dangling pointer, which is what lets a poll be a local.

8.3.3 Member Function Documentation

8.3.3.1 add()

```
template<typename asyncexecT>
void untangle::async::execution_poll::add (
                asyncexecT & async_exec)   [inline]
```

Adds an execution object to the poll.

Parameters

async_exec	An execution object.

8.3.3.2 remove()

```
template<typename asyncexecT>
void untangle::async::execution_poll::remove (
                asyncexecT & async_exec)   [inline]
```

Removes an execution object from the poll.

Attention

add() stores a pointer into the object, so one destroyed while still added would leave the poll calling into freed memory. ~execution() calls this.

Parameters

async_exec	An execution object.

8.3.3.3 withdraw()

```
void untangle::async::execution_poll::withdraw (
                const std::function< bool(void)> * action)   [inline]
```

Takes one action pointer out, without touching the execution it came from.

What ~execution() calls for each poll holding it: by then the object is being destroyed and must not be called back into, so the two halves of remove() are separated here.

Parameters

action	- The address add() stored.

8.3.3.4 is_running()

auto untangle::async::execution_poll::is_running () [inline]

Checks if the execution objects are running.

Returns

true - if at least one execution object in this poll is running.

The documentation for this class was generated from the following file:

- async.hpp

8.4 untangle::async::invalid_attachment Struct Reference

Raised by execution::attach() when the attachment asked for is refused.

#include <async.hpp>

Public Member Functions

- invalid_attachment (std::string text)
  Construct a new invalid attachment object.
- const char * what () const noexcept override
  The message text describing the reason of this exception.

8.4.1 Detailed Description

Raised by execution::attach() when the attachment asked for is refused.

Remarks

Not an untangle::invalid_action: that one reports a binding whose target has died and is swallowed by the worker, while this is a caller error and must not be.

8.4.2 Constructor & Destructor Documentation

8.4.2.1 invalid_attachment()

untangle::async::invalid_attachment::invalid_attachment (
std::string text) [inline], [explicit]

Construct a new invalid attachment object.

Parameters

text	- A message text, describing the reason of this exception.

8.4.3 Member Function Documentation

8.4.3.1 what()

const char * untangle::async::invalid_attachment::what () const [inline], [override], [noexcept]

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- async.hpp

Chapter 9

File Documentation

9.1 async.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #ifndef UNTANGLE_ASYNC_ASYNC_HPP
00007 #define UNTANGLE_ASYNC_ASYNC_HPP
00008
00009 #include <actuator/actuator.hpp>
00010 #include <algorithm>
00011 #include <atomic>
00012 #include <chrono>
00013 #include <condition_variable>
00014 #include <cstddef>
00015 #include <exception>
00016 #include <functional>
00017 #include <iterator>
00018 #include <memory>
00019 #include <mutex>
00020 #include <print>
00021 #include <string>
00022 #include <thread>
00023 #include <type_traits>
00024 #include <utility>
00025 #include <vector>
00026
00027 #ifdef _MSC_VER
00028 #include <intrin.h>
00029 #endif
00030
00031 namespace untangle {
00032 namespace async {
00033
00035 inline void cpu_pause() noexcept {
00036 #if defined(_MSC_VER) && (defined(_M_X64) || defined(_M_IX86))
00037   _mm_pause();
00038 #elif defined(_MSC_VER) && defined(_M_ARM64)
00039   __yield();
00040 #elif defined(__x86_64__) || defined(__i386__)
00041   __builtin_ia32_pause();
00042 #elif defined(__aarch64__) || defined(__arm__)
00043   __asm__ __volatile__("yield");
00044 #endif
00045 }
00046
00055 inline std::unique_lock<std::mutex> lock_spinning(std::mutex& mutex) {
00056   constexpr int tries = 100;
00057   for (int attempt = 0; attempt < tries; ++attempt) {
00058     if (mutex.try_lock()) {
00059       return std::unique_lock<std::mutex>(mutex, std::adopt_lock);
00060     }
00061     cpu_pause();
00062   }
00063   return std::unique_lock<std::mutex>(mutex);
00064 }
00068
00075 struct invalid_attachment : std::exception {
00081   explicit invalid_attachment(std::string text) : message_(std::move(text)) {}
00082
00088   const char* what() const noexcept override { return message_.c_str(); }
```

```
00089
00090  private:
00091   std::string message__;
00092 };
00093
00102 struct attachment {
00104   std::weak_ptr<attachment> attacher;
00105 };
00106
00117 class execution_poll {
00118  public:
00122   execution_poll() = default;
00123
00132   ~execution_poll() {
00133     std::vector<registration> held;
00134
00135     {
00136       std::lock_guard<std::mutex> lock(actuator_mutex__);
00137       held.swap(registrations__);
00138     }
00139
00140     // Outside the lock: each of these takes the execution's own, and ~execution() takes them in
00141     // the opposite order. Releasing this one first is what keeps the two from meeting in a cycle.
00142     for (auto& one : held) {
00143       one.forget();
00144     }
00145   }
00146
00152   template <typename asyncexecT>
00153   void add(asyncexecT& async_exec) {
00154     {
00155       std::lock_guard<std::mutex> lock(actuator_mutex__);
00156
00157       if (!actuator_is_running__.is_connected()) {
00158         actuator_is_running_ = untangle::connect(async_exec.action_is_running);
00159       } else {
00160         actuator_is_running__.add(&async_exec.action_is_running);
00161       }
00162
00163       registrations__.emplace_back(&async_exec.action_is_running,
00164                                    [this, &async_exec] { async_exec.forget_poll(*this); });
00165     }
00166
00167     // Outside the lock, and deliberately: this takes the execution's own. The execution has to
00168     // know which polls hold a pointer into it, because ~execution() is what takes it back out and
00169     // there is now more than one poll it could be in.
00170     async_exec.remember_poll(*this);
00171   }
00172
00181   template <typename asyncexecT>
00182   void remove(asyncexecT& async_exec) {
00183     withdraw(&async_exec.action_is_running);
00184     async_exec.forget_poll(*this);
00185   }
00186
00195   void withdraw(const std::function<bool(void)>* action) {
00196     std::lock_guard<std::mutex> lock(actuator_mutex__);
00197
00198     actuator_is_running__.remove(action);
00199     std::erase_if(registrations__,
00200                   [action](const registration& one) { return one.action == action; });
00201   }
00202
00208   auto is_running() {
00209     std::lock_guard<std::mutex> lock(actuator_mutex__);
00210     actuator_is_running__();
00211
00212     auto result = false;
00213     for (const auto& ret : actuator_is_running__.results) {
00214       result = result || ret;
00215     }
00216     return result;
00217   }
00218
00219  private:
00224   struct registration {
00225     const std::function<bool(void)>* action;
00226     std::function<void()> forget;
00227   };
00228
00229   actuator<std::function<bool(void)» actuator_is_running__;
00230   std::vector<registration> registrations__;
00231   mutable std::mutex actuator_mutex__;
00232 };
00233
00253 template <typename actionT>
00254 class execution {
```

```
00255   // An execution may attach one of a different specialisation, which is a different class;
00256   // attach() and detach() reach into its attachment_lifetime_.
00257   template <typename otherActionT>
00258   friend class execution;
00259
00260   // add() and remove() keep the record this object holds of the polls that hold it.
00261   friend class execution_poll;
00262
00263  public:
00273   template <typename... Args>
00274   static std::shared_ptr<execution> create_instance(Args&&... args) {
00275     return std::make_shared<execution>(std::forward<Args>(args)...);
00276   }
00277
00283   execution() : execution(std::string(default_name)) {}
00284
00290   explicit execution(std::string exec_name) : name(std::move(exec_name)) {
00291     action_execute = untangle::bind(this, &execution<actionT>::execute_actions);
00292     action_stop = untangle::bind(this, &execution<actionT>::stop);
00293     action_is_running = untangle::bind(this, &execution<actionT>::is_running);
00294   }
00295
00310   ~execution() {
00311     {
00312       auto lock = lock_spinning(action_mutex_);
00313       started_ = false;
00314       stopped_ = true;
00315     }
00316
00317     action_cv_.notify_all();
00318
00319     while (running_.load()) {  // time of check
00320       std::this_thread::sleep_for(std::chrono::milliseconds(1));
00321     }
00322
00323     // time of use: this object is freed once the destructor returns, so the check above is only
00324     // safe because running_ is the last thing the worker touches.
00325     //
00326     // Out of every poll that holds a pointer into this object: an execution can be in several,
00327     // and an address left in any of them outlives the object. Taken by swap so nothing else can be
00328     // added to the record while this walks it, and withdrawn rather than removed because remove()
00329     // would call back into an object that is going away.
00330     std::vector<execution_poll*> holding_polls;
00331
00332     {
00333       std::lock_guard<std::mutex> lock(polls_mutex_);
00334       holding_polls.swap(polls_);
00335     }
00336
00337     for (auto* poll : holding_polls) {
00338       poll->withdraw(&action_is_running);
00339     }
00340   }
00341
00359   template <typename classT, typename T>
00360   static void bind_action_and_method(actionT& action, const std::shared_ptr<classT>& obj,
00361                                      T classT::* method,
00362                                      const std::shared_ptr<execution>& async_exec) {
00363     auto async_action = std::make_shared<const actionT>(untangle::bind(obj, method));
00364
00365     action = [wp = std::weak_ptr<execution>(async_exec),
00366               async_action](auto&&... args) -> actionT::result_type {
00367       // lock() also keeps the execution alive for the duration of the call
00368       const auto exec = wp.lock();
00369       if (!exec) {
00370         throw invalid_action("bind: invalid execution");
00371       }
00372
00373       exec->add_queued_action([async_action, ... args = std::forward<decltype(args)>(args)] {
00374         return (*async_action)(args...);
00375       });
00376
00377       return typename actionT::result_type();
00378     };
00379   }
00380
00392   template <typename T>
00393   static void bind_action_and_function(actionT& action, T Fn,
00394                                        const std::shared_ptr<execution>& async_exec) {
00395     auto async_action = std::make_shared<const actionT>(std::move(Fn));
00396
00397     action = [wp = std::weak_ptr<execution>(async_exec),
00398               async_action](auto&&... args) -> actionT::result_type {
00399       const auto exec = wp.lock();
00400       if (!exec) {
00401         throw invalid_action("bind: invalid execution");
00402       }
```

```
00403
00404       exec->add_queued_action([async_action, ... args = std::forward<decltype(args)>(args)] {
00405         return (*async_action)(args...);
00406       });
00407
00408       return typename actionT::result_type();
00409     };
00410   }
00411
00423   bool is_running() const { return running_.load(); }
00424
00440   bool is_busy() const {
00441     auto lock = lock_spinning(action_mutex_);
00442     return has_pending_actions_or_tasks() || executing_action_.load();
00443   }
00444
00453   std::size_t pending() const {
00454     auto lock = lock_spinning(action_mutex_);
00455     return action_actuator_.actions.size() + action_actuator_.actions_map.size() +
00456            action_actuator_.tasks.size();
00457   }
00458
00472   void run() {
00473     wait_thread_to_finish();
00474     collecting_results_ = true;
00475     thread_ = std::thread(&execution::execute, this);
00476     thread_.detach();
00477   }
00478
00489   void start() {
00490     wait_thread_to_finish();
00491     collecting_results_ = false;
00492     {
00493       auto lock = lock_spinning(action_mutex_);
00494       started_ = true;
00495       stopped_ = false;
00496     }
00497
00498     thread_ = std::thread(&execution::loop, this);
00499     thread_.detach();
00500   }
00501
00508   void stop() {
00509     {
00510       auto lock = lock_spinning(action_mutex_);
00511       started_ = false;
00512       stopped_ = true;
00513     }
00514
00515     // The worker drains what is already queued before it leaves loop(), so stop() does not have to
00516     // spin on the queue; waking it is enough.
00517     action_cv_.notify_all();
00518
00519     if (actuator_stop_.is_connected()) {
00520       actuator_stop_();
00521     }
00522   }
00523
00551   template <typename... Args>
00552   bool add_action(actionT action, Args&&... args) {
00553     return add_queued_action(std::bind(std::move(action), std::forward<Args>(args)...));
00554   }
00555
00588   template <typename... Args>
00589   bool add_task(const actionT& action, Args&&... args) {
00590     return add_queued_task(untangle::bind_task(action, std::forward<Args>(args)...));
00591   }
00592
00597   template <typename... Args>
00598   bool add_task(actionT&& action, Args&&... args) {
00599     return add_queued_task(untangle::bind_task(std::move(action), std::forward<Args>(args)...));
00600   }
00601
00615   bool add_queued_task(untangle::task_t task) {
00616     bool wake = false;
00617     {
00618       auto lock = lock_spinning(action_mutex_);
00619
00620       // As for an action: a stopped worker would never reach it, and a task left in the queue
00621       // would look pending while being unable to ever notify.
00622       if (stopped_) {
00623         std::println(stderr, "warning: execution '{}' is stopped, task not added", name);
00624         return false;
00625       }
00626
00627       if (!action_actuator_.add_task(std::move(task))) {
00628         std::println(stderr, "warning: execution '{}' refused a task that cannot report", name);
```

```
00629       return false;
00630     }
00631     wake = sleeping_;
00632   }
00633
00634   // A worker that has not parked finds the work on its own.
00635   if (wake) {
00636     action_cv_.notify_one();
00637   }
00638   return true;
00639 }
00640
00659 template <typename otherT>
00660 void attach(const std::shared_ptr<otherT>& other) {
00661   if (static_cast<const void*>(other.get()) == static_cast<const void*>(this)) {
00662     throw invalid_attachment("attach: an execution cannot be attached to itself");
00663   }
00664
00665   if (!other->attachment_lifetime_->attacher.expired()) {
00666     throw invalid_attachment("attach: that execution is attached already");
00667   }
00668
00669   // At most one attacher each makes the graph a forest, so a cycle closes exactly when other is
00670   // somewhere up this chain. Walking it is the whole check.
00671   for (auto link = attachment_lifetime_->attacher.lock(); link; link = link->attacher.lock()) {
00672     if (link == other->attachment_lifetime_) {
00673       throw invalid_attachment("attach: the attachment would close a cycle");
00674     }
00675   }
00676
00677   // Bound weakly, as owned actions: a call keeps other alive, a call to one that is gone throws
00678   // untangle::invalid_action, and the actuator drops it.
00679   std::erase_if(attachments_, [](const auto& entry) { return entry.attached.expired(); });
00680   attachments_.push_back({other->attachment_lifetime_,
00681                           actuator_execute_.add(untangle::bind(other, &otherT::execute_actions)),
00682                           actuator_stop_.add(untangle::bind(other, &otherT::stop))});
00683   other->attachment_lifetime_->attacher = attachment_lifetime_;
00684
00685   // An idle worker blocks until it is told otherwise; from now on it has to poll.
00686   {
00687     auto lock = lock_spinning(action_mutex_);
00688     attached_ = true;
00689   }
00690   action_cv_.notify_all();
00691 }
00692
00708 template <typename otherT>
00709 bool detach(otherT& other) {
00710   if (actuator_execute_.is_dispatching() || actuator_stop_.is_dispatching()) {
00711     return false;
00712   }
00713
00714   const auto entry = std::ranges::find_if(attachments_, [&other](const auto& candidate) {
00715     return candidate.attached.lock() == other.attachment_lifetime_;
00716   });
00717   if (entry == attachments_.end()) {
00718     return false;
00719   }
00720
00721   actuator_execute_.remove(entry->execute);
00722   actuator_stop_.remove(entry->stop);
00723   attachments_.erase(entry);
00724
00725   other.attachment_lifetime_->attacher.reset();
00726
00727   {
00728     auto lock = lock_spinning(action_mutex_);
00729     attached_ = actuator_execute_.is_connected();
00730   }
00731
00732   return true;
00733 }
00734
00746 auto results() const {
00747   std::lock_guard<std::mutex> lock(results_mutex_);
00748   return results_;
00749 }
00750
00751 static constexpr auto default_name = "default_name";
00752
00758 std::function<void(void)> action_execute;
00759
00765 std::function<void(void)> action_stop;
00766
00775 std::function<bool(void)> action_is_running;
00776
00788 std::function<void(void)> on_finished;
```

```
00789
00815   std::function<void(std::exception_ptr)> on_error;
00816
00817   std::string name = default_name;
00818
00819  private:
00820   using queued_action_t = std::function<typename actionT::result_type(void)>;
00821
00826   void remember_poll(execution_poll& poll) {
00827     std::lock_guard<std::mutex> lock(polls_mutex_);
00828
00829     polls_.push_back(&poll);
00830   }
00831
00835   void forget_poll(execution_poll& poll) {
00836     std::lock_guard<std::mutex> lock(polls_mutex_);
00837
00838     std::erase(polls_, &poll);
00839   }
00840
00850   bool has_pending_actions_or_tasks() const {
00851     return action_actuator_.is_connected() || action_actuator_.has_tasks();
00852   }
00853
00863   bool add_queued_action(queued_action_t action) {
00864     bool wake = false;
00865     {
00866       auto lock = lock_spinning(action_mutex_);
00867
00868       // Once stopped, the worker is on its way out and would never reach this action; dropping it
00869       // here is what keeps it from sitting in the queue looking as though it were pending.
00870       if (stopped_) {
00871         std::println(stderr, "warning: execution '{}' is stopped, action not added", name);
00872         return false;
00873       }
00874
00875       action_actuator_.add(std::move(action));
00876       wake = sleeping_;
00877     }
00878
00879     // A worker that has not parked finds the work on its own.
00880     if (wake) {
00881       action_cv_.notify_one();
00882     }
00883     return true;
00884   }
00885
00895   void execute_actions() {
00896     actions_and_tasks_run_ = 0;
00897
00898     for (;;) {
00899       // Takes over action_actuator_ under the lock; its actions then run without it.
00900       actuator<queued_action_t> batch;
00901
00902       {
00903         auto lock = lock_spinning(action_mutex_);
00904         if (!has_pending_actions_or_tasks()) {
00905           break;
00906         }
00907
00908         batch = std::move(action_actuator_);
00909
00910         // Under the same lock, so is_busy() sees both or neither.
00911         executing_action_ = true;
00912       }
00913
00914       // Counted before either call: a dead binding leaves the actions list during operator(),
00915       // and call_tasks() consumes the tasks list outright.
00916       const auto in_the_batch = batch.actions.size() + batch.tasks.size();
00917
00918       // The actions have their own arms inside the actuator; this one is for the work around
00919       // them, and for the rule that nothing may leave this loop: an exception escaping a detached
00920       // thread function calls std::terminate.
00921       try {
00922         batch();
00923
00924         // Empty for a void action type, which is what makes this one line rather than two arms.
00925         if (collecting_results_) {
00926           std::lock_guard<std::mutex> lock(results_mutex_);
00927           std::ranges::move(batch.results, std::back_inserter(results_));
00928         }
00929
00930         // After the actions, and deliberately: one pass fires every action it holds and then every
00931         // task, so within a batch the two kinds do not interleave in the order they were queued.
00932         // Across batches the queue is still first in, first out. A task reports to its own
00933         // callback, so nothing is collected here; what one throws lands in batch.errors below,
00934         // beside what an action threw.
```

```
00935       batch.call_tasks();
00936     } catch (...) {
00937       report_error(std::current_exception());
00938     }
00939
00940     for (const auto& thrown : batch.errors) {
00941       report_error(thrown);
00942     }
00943
00944     actions_and_tasks_run_ += in_the_batch;
00945
00946     executing_action_ = false;
00947
00948     // The notification is raised without the lock: on_finished may call add_action(), which
00949     // takes it.
00950     bool drained = false;
00951     {
00952       auto lock = lock_spinning(action_mutex_);
00953       drained = !has_pending_actions_or_tasks();
00954     }
00955
00956     if (drained) {
00957       notify_finished();
00958     }
00959   }
00960
00961   // Last, and part of the pass: an attachment is only ever reached through execute_actions(),
00962   // which is this function, so whoever drives this execution drives the ones attached to it.
00963   if (actuator_execute_.is_connected()) {
00964     actuator_execute_();
00965
00966     // The pass dropped the last attachment, gone since: the worker can block again.
00967     if (!actuator_execute_.is_connected()) {
00968       auto lock = lock_spinning(action_mutex_);
00969       attached_ = false;
00970     }
00971   }
00972 }
00973
00979 void report_error(std::exception_ptr thrown) {
00980   if (on_error) {
00981     // The handler is the caller's code, on a detached thread; anything escaping it would leave
00982     // the thread function and call std::terminate.
00983     try {
00984       on_error(thrown);
00985     } catch (...) {
00986       std::println(stderr,
00987                    "warning: execution '{}' had on_error throw, and dropped what it threw", name);
00988     }
00989
00990     return;
00991   }
00992
00993   try {
00994     std::rethrow_exception(thrown);
00995   } catch (const invalid_action& ia) {
00996     std::println(stderr, "warning: execution '{}' dropped an invalid action: {}", name,
00997                  ia.what());
00998   } catch (const std::exception& e) {
00999     std::println(stderr, "warning: execution '{}' dropped an action that threw: {}", name,
01000                  e.what());
01001   } catch (...) {
01002     std::println(stderr, "warning: execution '{}' dropped an action that threw an unknown type",
01003                  name);
01004   }
01005 }
01006
01014 void notify_finished() {
01015   if (actions_and_tasks_run_ == 0 || !on_finished) {
01016     return;
01017   }
01018
01019   {
01020     // An action may queue another, so a pass that drained can leave more behind it. That is the
01021     // next batch, not the end of this one.
01022     auto lock = lock_spinning(action_mutex_);
01023     if (has_pending_actions_or_tasks()) {
01024       return;
01025     }
01026   }
01027
01028   on_finished();
01029 }
01030
01038 void wait_thread_to_finish() {
01039   auto resident = false;
01040   while (!running_.compare_exchange_weak(resident, true)) {
```

```
01041       resident = false;
01042       std::this_thread::sleep_for(std::chrono::milliseconds(1));
01043     }
01044   }
01045
01046   void execute() {
01047     // The results belong to this run, not to this run and the last one.
01048     {
01049       std::lock_guard<std::mutex> lock(results_mutex_);
01050       results_.clear();
01051     }
01052
01053     execute_actions();
01054
01055     // Must stay last: ~execution() may free this object the moment it reads false.
01056     running_ = false;
01057   }
01058
01059   void loop() {
01060     for (;;) {
01061       {
01062         auto lock = lock_spinning(action_mutex_);
01063
01064         const auto has_work = [this] { return has_pending_actions_or_tasks() || !started_; };
01065
01066         // Polls only while something is attached: an attached execution has its own list and
01067         // cannot notify this condition variable. With nothing attached the worker blocks until it
01068         // has work, is stopped, or attach() wakes it.
01069         // Parked in either wait, and under the lock: an add reads it under the same lock, so it
01070         // notifies exactly when the worker may be waiting.
01071         sleeping_ = true;
01072         if (attached_) {
01073           action_cv_.wait_for(lock, std::chrono::milliseconds(10), has_work);
01074         } else {
01075           action_cv_.wait(lock, [this, &has_work] { return has_work() || attached_; });
01076         }
01077         sleeping_ = false;
01078
01079         if (!started_ && !has_pending_actions_or_tasks()) {
01080           break;
01081         }
01082       }
01083       execute_actions();
01084     }
01085
01086     // A last pass for the attachments; this execution's own list is already empty here.
01087     execute_actions();
01088
01089     // Must stay last: ~execution() may free this object the moment it reads false.
01090     running_ = false;
01091   }
01092
01093   std::thread thread_;
01094
01095   std::atomic_bool started_ = {false};
01096   std::atomic_bool running_ = {false};
01097
01098   // Set by stop(), and the difference between "not started yet" and "finished for good": actions
01099   // may be queued before run()/start(), but not after stop().
01100   std::atomic_bool stopped_ = {false};
01101
01102   // action_actuator_, started_ and stopped_ are written by every thread that calls add_action() or
01103   // stop() and read by the worker; nothing touches them outside this mutex.
01104   mutable std::mutex action_mutex_;
01105   std::condition_variable action_cv_;
01106
01108   bool sleeping_ = false;
01109
01118   bool attached_ = false;
01119
01125   actuator<queued_action_t> action_actuator_;
01126
01135   using void_actuator = actuator<std::function<void(void)>>;
01136
01137   void_actuator actuator_execute_;
01138   void_actuator actuator_stop_;
01139
01151   std::shared_ptr<attachment> attachment_lifetime_ = std::make_shared<attachment>();
01152
01160   struct attached_entry {
01161     std::weak_ptr<attachment> attached;
01162     const std::function<void(void)>* execute = nullptr;
01163     const std::function<void(void)>* stop = nullptr;
01164   };
01165   std::vector<attached_entry> attachments_;
01166
01167   // std::vector cannot hold void type; use an arbitrary type e.g. int
```

```
01168   using resultT = std::conditional<std::is_void<typename actionT::result_type>::value, int,
01169                                    typename actionT::result_type>;
01170
01174   std::vector<typename resultT::type> results_;
01175
01179   mutable std::mutex results_mutex_;
01180
01187   std::atomic_bool collecting_results_ = {false};
01188
01194   std::atomic_bool executing_action_ = {false};
01195
01197   std::vector<execution_poll*> polls_;
01198   mutable std::mutex polls_mutex_;
01199
01211   std::atomic_size_t actions_and_tasks_run_ = {0};
01212 };
01213 }  // namespace async
01214 }  // namespace untangle
01215
01216 #endif  // UNTANGLE_ASYNC_ASYNC_HPP
```

Index

~execution
untangle::async::execution< actionT >, 22
~execution_poll
untangle::async::execution_poll, 32

action_execute
untangle::async::execution< actionT >, 30
action_is_running
untangle::async::execution< actionT >, 30
action_stop
untangle::async::execution< actionT >, 30
add
untangle::async::execution_poll, 33
add_action
untangle::async::execution< actionT >, 26
add_queued_task
untangle::async::execution< actionT >, 28
add_task
untangle::async::execution< actionT >, 27
async, 1
attach
untangle::async::execution< actionT >, 28

bind_action_and_function
untangle::async::execution< actionT >, 23
bind_action_and_method
untangle::async::execution< actionT >, 23

create_instance
untangle::async::execution< actionT >, 22

detach
untangle::async::execution< actionT >, 29

execution
untangle::async::execution< actionT >, 21

invalid_attachment
untangle::async::invalid_attachment, 34
is_busy
untangle::async::execution< actionT >, 24
is_running
untangle::async::execution< actionT >, 24
untangle::async::execution_poll, 34

namespace untangle: functions, 15

on_error
untangle::async::execution< actionT >, 31
on_finished
untangle::async::execution< actionT >, 31

pending
untangle::async::execution< actionT >, 24

remove
untangle::async::execution_poll, 33
results
untangle::async::execution< actionT >, 29
run
untangle::async::execution< actionT >, 25

start
untangle::async::execution< actionT >, 25
stop
untangle::async::execution< actionT >, 25

untangle, 17
untangle::async::attachment, 19
untangle::async::execution< actionT >, 19
~execution, 22
action_execute, 30
action_is_running, 30
action_stop, 30
add_action, 26
add_queued_task, 28
add_task, 27
attach, 28
bind_action_and_function, 23
bind_action_and_method, 23
create_instance, 22
detach, 29
execution, 21
is_busy, 24
is_running, 24
on_error, 31
on_finished, 31
pending, 24
results, 29
run, 25
start, 25
stop, 25
untangle::async::execution_poll, 32
~execution_poll, 32
add, 33
is_running, 34
remove, 33
withdraw, 33
untangle::async::invalid_attachment, 34
invalid_attachment, 34
what, 35

what

untangle::async::invalid_attachment, 35
withdraw
untangle::async::execution_poll, 33